

WEEK 8 – Transactional Microservice

Spring Boot + Spring Data JPA + PostgreSQL · Inventory and Warehouse Management System

Team 17 · Section 12 · M. Laasya Sri (2520090196), M. Rajini (2520090197), K. Ramya (2520090313)

Database Systems Engineering and Distributed Backend Development (25CS1302E) · KLH Bachupally Campus

Aim

Warehouse stock and order operations need financial-style accuracy: stock must never go below zero, and an order must either fully succeed or fully fail. To achieve this we use Spring Boot and PostgreSQL. You will build a transactional microservice for the Inventory and Warehouse Management System and connect it to the existing FastAPI gateway.

Scenario

The Inventory and Warehouse Management System needs a strict service to manage **Inventory** (how many units of each product are stored in a warehouse slot) and **Orders** (which staff member took how many units of which product, and when). Because these operations involve exact quantities, the service uses Spring Boot with Spring Data JPA, and PostgreSQL as the database.

Tasks to Be Done

1. **Spring Boot project setup:** use Spring Initializr to create a Maven project with Spring Web, Spring Data JPA, PostgreSQL Driver and Validation. Configure application.yml to connect to your PostgreSQL instance.
2. **Entity mapping (the relational model):** create an *Inventory* entity to track productId (from the FastAPI product service), warehouseId, capacity, quantity and slotCode. Create a *StockOrder* entity to track userId (the staff member from the auth service), productId, quantity and status (PLACED / CANCELLED). Use JPA annotations (@Entity, @Table, @Id, @Column) to map them to tables.
3. **Service layer (CRUD and logic):** create repository interfaces extending JpaRepository for zero-code CRUD. In the service, check that the requested quantity is available before allowing an order. Implement @PostMapping for /orders and @GetMapping for /inventory/{productId}.
4. **Validation and error handling:** use JSR-380 annotations (such as @Min(0)) so stock never goes below zero. Implement a @RestControllerAdvice class that returns a clean JSON error when a product is out of stock.
5. **Gateway integration:** update your FastAPI gateway to route requests to this new service.

The final test: a staff user places an order through the gateway → the gateway calls Spring Boot → Spring Boot checks stock and records the order.

Skill Set Gained

- **Strict layering:** the industry-standard Controller–Service–Repository pattern.

- **Type safety:** Java's strong typing and Spring's Dependency Injection (IoC) for reliable code.
- **Automated persistence:** Spring Data JPA talks to SQL without hand-written queries.
- **Polyglot harmony:** Python (FastAPI) and Java (Spring Boot) working together in one system.

Solution

1. Project Objective

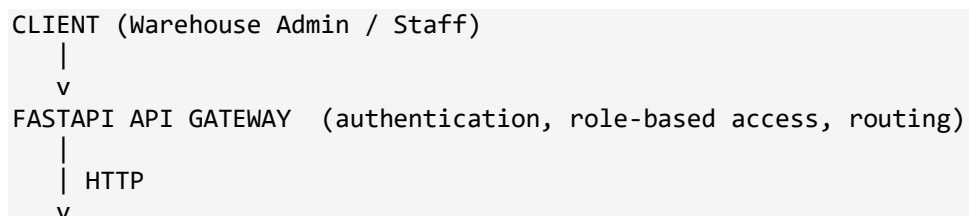
Build a Spring Boot microservice that:

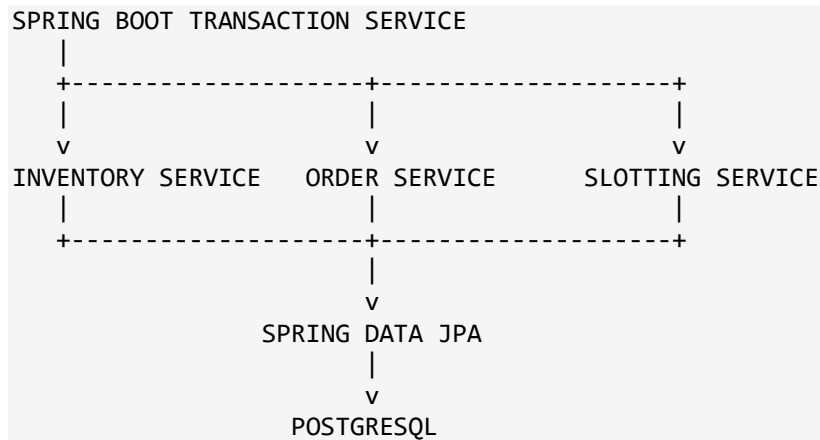
- manages stock per product and warehouse;
- records stock orders and reduces inventory inside one database transaction;
- prevents negative stock with JSR-380 validation;
- returns clean JSON errors through @RestControllerAdvice;
- suggests warehouse slots with DSA (priority queue + greedy) for Warehouse Slotting Optimization;
- is reached through the FastAPI gateway.

2. Technology Stack

Technology	Purpose
Java 17+	Application language
Spring Boot	Microservice framework
Spring Web	REST APIs
Spring Data JPA	Database persistence
Validation (JSR-380)	Input and stock-level checks
PostgreSQL	Relational database
Maven	Build and dependency management
Postman	API testing
FastAPI	Gateway integration
VS Code / IntelliJ / Eclipse	Development IDE

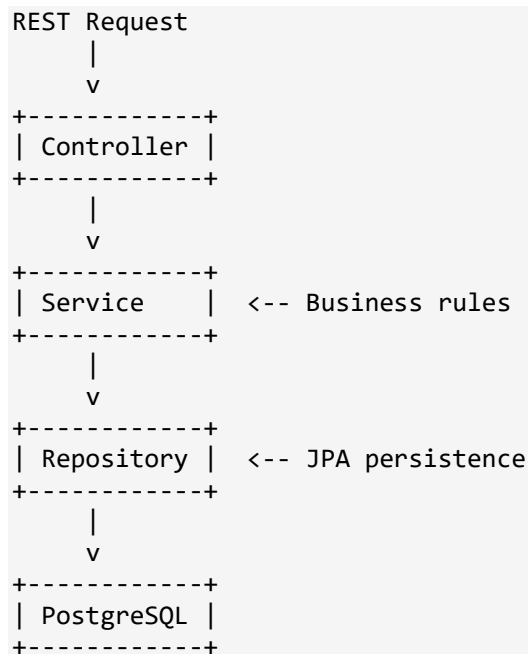
3. Architecture Dashboard





The Spring Boot service checks available stock, creates an order, and reduces inventory within a single database transaction. PostgreSQL stores the persistent data.

4. Internal Layered Architecture



5. Project Structure

```

iwms-transaction/
├── pom.xml
├── src/main/
│   ├── java/com/iwms/transaction/
│   │   ├── IwmsTransactionApplication.java
│   │   ├── controller/
│   │   │   ├── InventoryController.java
│   │   │   ├── OrderController.java
│   │   │   └── SlottingController.java
│   │   ├── entity/
│   │   └── Inventory.java
  
```

```

├── StockOrder.java
├── repository/
│   ├── InventoryRepository.java
│   └── StockOrderRepository.java
├── service/
│   ├── InventoryService.java
│   ├── OrderService.java
│   └── SlottingService.java
├── dto/
│   ├── OrderRequest.java
│   └── SlotAssignment.java
├── exception/
│   ├── OutOfStockException.java
│   ├── ProductNotFoundException.java
│   └── GlobalExceptionHandler.java
└── resources/
    └── application.yml

```

6. Database Model

INVENTORY

```

-----
product_id    PRIMARY KEY
warehouse_id  NOT NULL
capacity      >= 0
quantity      >= 0
slot_code
demand_score  >= 0

```

STOCK_ORDERS

```

-----
id            PRIMARY KEY
user_id       NOT NULL
product_id    NOT NULL
quantity      >= 1
status        PLACED / CANCELLED

```

7. PostgreSQL Setup

```
CREATE DATABASE iwms_week8;
```

```
-- After connecting to iwms_week8:
```

```
SELECT current_database();
```

Expected: iwms_week8

8. application.yml

```

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/iwms_week8
    username: postgres
    password: postgres
  jpa:
    hibernate:

```

```
ddl-auto: update
show-sql: true
properties:
  hibernate:
    format_sql: true
server:
  port: 8080
```

9. Main Program

```
package com.iwms.transaction;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class IwmsTransactionApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            IwmsTransactionApplication.class, args);
    }
}
```

10. Inventory Entity

In Spring Boot 3 the annotations come from the `jakarta` packages. Generate constructors, getters and setters with your IDE (Source → Generate Getters and Setters).

```
package com.iwms.transaction.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;

@Entity
@Table(name = "inventory")
public class Inventory {

    @Id
    private Long productId;

    @NotNull
    private Long warehouseId;

    @Min(0)
    private Integer capacity;

    @Min(0)
    private Integer quantity;

    private String slotCode;

    @Min(0)
    private Integer demandScore;
```

```

    // constructors, getters and setters
}

```

11. StockOrder Entity

```

package com.iwms.transaction.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;

@Entity
@Table(name = "stock_orders")
public class StockOrder {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotNull
    private Long userId;

    @NotNull
    private Long productId;

    @Min(1)
    private Integer quantity;

    private String status;

    // constructors, getters and setters
}

```

12. Repository Programs

```

public interface StockOrderRepository
    extends JpaRepository<StockOrder, Long> {
}

```

```

public interface InventoryRepository
    extends JpaRepository<Inventory, Long> {

    // Locks the row until the transaction ends, so two
    // simultaneous orders cannot oversell the same product.
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select i from Inventory i where i.productId = :id")
    Optional<Inventory> findForUpdate(@Param("id") Long id);
}

```

JpaRepository supplies save(), findById(), findAll(), delete(), existsById() and other CRUD operations.

13. Order Request DTO

```

package com.iwms.transaction.dto;

```

```

import jakarta.validation.constraints.*;

public class OrderRequest {

    @NotNull
    private Long userId;

    @NotNull
    private Long productId;

    @NotNull
    @Min(1)
    private Integer quantity;

    // getters and setters
}

```

14. Transactional Service Logic

```

@Service
public class OrderService {

    private final InventoryRepository inventoryRepository;
    private final StockOrderRepository orderRepository;

    public OrderService(InventoryRepository inventoryRepository,
                        StockOrderRepository orderRepository) {
        this.inventoryRepository = inventoryRepository;
        this.orderRepository = orderRepository;
    }

    @Transactional
    public StockOrder placeOrder(OrderRequest request) {

        Inventory inventory = inventoryRepository
            .findForUpdate(request.getProductId())
            .orElseThrow(() ->
                new ProductNotFoundException(
                    "Product " + request.getProductId() + " not found"));

        if (inventory.getQuantity() < request.getQuantity()) {
            throw new OutOfStockException(
                "Product " + request.getProductId()
                + " has only " + inventory.getQuantity()
                + " units available");
        }

        inventory.setQuantity(
            inventory.getQuantity() - request.getQuantity());
        inventoryRepository.save(inventory);

        StockOrder order = new StockOrder(
            request.getUserId(),
            request.getProductId(),
            request.getQuantity(),

```

```

        "PLACED");
        return orderRepository.save(order);
    }
}

```

Because the method is `@Transactional`, the stock reduction and the order insert commit together. If either fails, both are rolled back.

```

@Service
public class InventoryService {

    private final InventoryRepository inventoryRepository;

    public InventoryService(InventoryRepository inventoryRepository) {
        this.inventoryRepository = inventoryRepository;
    }

    public Inventory getInventory(Long productId) {
        return inventoryRepository.findById(productId)
            .orElseThrow(() ->
                new ProductNotFoundException(
                    "Product " + productId + " not found"));
    }
}

```

15. Controller Programs

```

@RestController
@RequestMapping("/inventory")
public class InventoryController {

    private final InventoryService inventoryService;

    public InventoryController(InventoryService inventoryService) {
        this.inventoryService = inventoryService;
    }

    @GetMapping("/{productId}")
    public Inventory getInventory(@PathVariable Long productId) {
        return inventoryService.getInventory(productId);
    }
}

```

```

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping

```

```

    @ResponseStatus(HttpStatus.CREATED)
    public StockOrder placeOrder(
        @Valid @RequestBody OrderRequest request) {
        return orderService.placeOrder(request);
    }
}

```

16. Global Error Handling

Create the two exception classes first (each extends RuntimeException and passes its message to super), then the handler.

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(OutOfStockException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public Map<String, Object> handleOutOfStock(
        OutOfStockException ex) {
        return Map.of(
            "status", 409,
            "error", "OUT_OF_STOCK",
            "message", ex.getMessage());
    }

    @ExceptionHandler(ProductNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, Object> handleNotFound(
        ProductNotFoundException ex) {
        return Map.of(
            "status", 404,
            "error", "PRODUCT_NOT_FOUND",
            "message", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public Map<String, Object> handleValidation(
        MethodArgumentNotValidException ex) {
        return Map.of(
            "status", 400,
            "error", "VALIDATION_FAILED",
            "message", "Quantity must be at least 1");
    }
}

```

17. Dashboard – Initial Database State

The following dashboard shows the expected PostgreSQL state after inserting the sample inventory.

Product ID	Warehouse	Capacity	In Stock	Slot	Status
101	1	100	50	A-01	AVAILABLE

Product ID	Warehouse	Capacity	In Stock	Slot	Status
102	1	80	30	B-04	AVAILABLE
103	2	50	0	C-09	OUT OF STOCK

Visual stock indicator (one block = 5 units):

```
Product 101  ██████████ 50 in stock
Product 102  ████████ 30 in stock
Product 103  0 in stock
```

18. Insert Sample Data

```
INSERT INTO inventory
(product_id, warehouse_id, capacity, quantity, slot_code, demand_score)
VALUES
(101, 1, 100, 50, 'A-01', 90),
(102, 1, 80, 30, 'B-04', 60),
(103, 2, 50, 0, 'C-09', 10);

SELECT * FROM inventory;
```

19. Dashboard – After One Order

After staff user 501 orders 5 units of product 101:

Metric	Before	After	Change
Product 101 capacity	100	100	0
Product 101 in stock	50	45	-5
Orders created	0	1	+1
Order status	none	PLACED	new

```
Product 101  ██████████ 45 in stock / 100 capacity
Orders       █ 1 order
```

20. How to Run the Project – Step by Step

1. **Install Java:** install JDK 17 or later and verify with `java -version`.
2. **Verify Maven:** run `mvn -version`.
3. **Start PostgreSQL:** make sure the PostgreSQL service is running.
4. **Create the database:** create a database named `iwms_week8`.
5. **Open the project:** open the generated Maven project in VS Code or IntelliJ.
6. **Configure the database:** edit `application.yml` with your PostgreSQL username and password.
7. **Build the project:** run `mvn clean install`.
8. **Start Spring Boot:** run `mvn spring-boot:run`.
9. **Check the server:** open `http://localhost:8080` and test with Postman.
10. **Insert test data:** insert the sample inventory using pgAdmin or psql.

11. **Test GET:** call GET /inventory/101.
12. **Test POST:** call POST /orders with userId, productId and quantity.
13. **Verify the database:** query the inventory and stock_orders tables.

21. Expected Console Output

```
Tomcat initialized with port 8080
Hibernate: create table inventory ...
Hibernate: create table stock_orders ...
Tomcat started on port 8080
Started IwmsTransactionApplication
```

22. Postman Testing – GET Inventory

Item	Value
Method	GET
URL	http://localhost:8080/inventory/101
Body	None

Expected response:

```
{
  "productId": 101,
  "warehouseId": 1,
  "capacity": 100,
  "quantity": 50,
  "slotCode": "A-01",
  "demandScore": 90
}
```

23. Postman Testing – Place Order

Item	Value
Method	POST
URL	http://localhost:8080/orders
Header	Content-Type: application/json

Body:

```
{
  "userId": 501,
  "productId": 101,
  "quantity": 5
}
```

Expected response (HTTP 201 Created):

```
{
  "id": 1,
```

```
"userId": 501,  
"productId": 101,  
"quantity": 5,  
"status": "PLACED"  
}
```

24. Verify Inventory After Order

Call GET `http://localhost:8080/inventory/101` again. The quantity must now be 45:

```
{  
  "productId": 101,  
  "warehouseId": 1,  
  "capacity": 100,  
  "quantity": 45,  
  "slotCode": "A-01",  
  "demandScore": 90  
}
```

25. Verify PostgreSQL

```
SELECT * FROM inventory;  
SELECT * FROM stock_orders;
```

product_id	warehouse_id	capacity	quantity
101	1	100	45
102	1	80	30
103	2	50	0

id	user_id	product_id	quantity	status
1	501	101	5	PLACED

26. Out-of-Stock Test

If the available quantity is lower than the requested quantity, the service must not create an order.

POST `http://localhost:8080/orders`

```
{  
  "userId": 502,  
  "productId": 103,  
  "quantity": 1  
}
```

Expected HTTP response: **409 Conflict**

```
{  
  "status": 409,  
  "error": "OUT_OF_STOCK",  
}
```

```

    "message": "Product 103 has only 0 units available"
}

```

27. Validation Test

Stock values must not be negative. @Min(0) rejects values below zero, and @Min(1) rejects an order quantity of 0.

```

@Min(value = 0, message = "Quantity cannot be negative")
private Integer quantity;
Send POST /orders with "quantity": 0. Expected: 400 Bad Request with VALIDATION_FAILED.

```

28. End-to-End Gateway Dashboard

```

CLIENT
|
| POST /gateway/orders
| { userId: 501, productId: 101, quantity: 5 }
v
FASTAPI GATEWAY
|
| HTTP POST /orders
v
SPRING BOOT
|
+--> Find inventory (row locked)
|
+--> quantity >= requested ?
|
|      +-- NO --> 409 OUT_OF_STOCK
|      |
|      +-- YES
|           |
|           +--> quantity - requested
|           +--> Create PLACED order
v
POSTGRESQL
|
v
JSON RESPONSE
{ id: 1, userId: 501, productId: 101, quantity: 5, status: "PLACED" }

```

29. FastAPI Gateway Example

```

from fastapi import FastAPI, HTTPException
import httpx

app = FastAPI()

SPRING_SERVICE = "http://localhost:8080"

@app.post("/gateway/orders")
async def create_order(order: dict):
    async with httpx.AsyncClient() as client:

```

```

        response = await client.post(
            f"{SPRING_SERVICE}/orders",
            json=order
        )
    if response.status_code >= 400:
        raise HTTPException(
            status_code=response.status_code,
            detail=response.json()
        )
    return response.json()

@app.get("/gateway/inventory/{product_id}")
async def get_inventory(product_id: int):
    async with httpx.AsyncClient() as client:
        response = await client.get(
            f"{SPRING_SERVICE}/inventory/{product_id}"
        )
    return response.json()

```

30. Extension – Warehouse Slotting Optimization (DSA)

The project abstract focuses on Warehouse Slotting Optimization. This extension ranks products by demand with a **priority queue** (max-heap) and assigns them **greedily**: the highest-demand product gets the free slot nearest the dock, the next product gets the next slot, and so on. Sorting plus heap operations cost $O(n \log n)$.

```
public record SlotAssignment(Long productId, String slotCode) {}
```

```

@Service
public class SlottingService {

    private final InventoryRepository inventoryRepository;

    public SlottingService(InventoryRepository inventoryRepository) {
        this.inventoryRepository = inventoryRepository;
    }

    // freeSlots must be ordered from nearest to farthest from the dock
    public List<SlotAssignment> suggestSlots(List<String> freeSlots) {

        PriorityQueue<Inventory> queue = new PriorityQueue<>(
            (a, b) -> Integer.compare(
                b.getDemandScore(), a.getDemandScore()));
        queue.addAll(inventoryRepository.findAll());

        List<SlotAssignment> plan = new ArrayList<>();
        for (String slot : freeSlots) {
            Inventory next = queue.poll();
            if (next == null) break;
            plan.add(new SlotAssignment(next.getProductId(), slot));
        }
        return plan;
    }
}

```

```
}  
}
```

```
@RestController  
@RequestMapping("/slotting")  
public class SlottingController {  
  
    private final SlottingService slottingService;  
  
    public SlottingController(SlottingService slottingService) {  
        this.slottingService = slottingService;  
    }  
  
    @GetMapping("/plan")  
    public List<SlotAssignment> plan(@RequestParam List<String> slots) {  
        return slottingService.suggestSlots(slots);  
    }  
}
```

Test: GET http://localhost:8080/slotting/plan?slots=A-01,A-02,A-03

```
[  
  { "productId": 101, "slotCode": "A-01" },  
  { "productId": 102, "slotCode": "A-02" },  
  { "productId": 103, "slotCode": "A-03" }  
]
```

To extend it, skip slots whose capacity is smaller than the product's capacity and respect product requirements (for example cold storage).

31. Complete Testing Checklist

Test	Expected Result	Status
Spring Boot starts	Port 8080 active	Pending
PostgreSQL connection	Application connects	Pending
GET inventory	Returns stock JSON	Pending
POST order	Creates PLACED order	Pending
Stock decrement	Quantity decreases by the ordered amount	Pending
Zero stock	HTTP 409 OUT_OF_STOCK, order rejected	Pending
Negative / zero quantity validation	HTTP 400 VALIDATION_FAILED	Pending
Database persistence	Rows remain in PostgreSQL	Pending
Gateway forwarding	FastAPI reaches Spring Boot	Pending

Test	Expected Result	Status
Slotting plan	Highest demand gets the first slot	Pending

32. Common Problems and Solutions

Problem	Likely Cause	Solution
Connection refused	PostgreSQL not running	Start the PostgreSQL service
Password authentication failed	Wrong DB password	Update application.yml
Port 8080 in use	Another service uses 8080	Stop it or change server.port
Product not found (404)	No inventory row	Insert the product into inventory
Out of stock (409)	quantity is lower than requested	Add stock or choose another product
Maven command not found	Maven not installed or PATH issue	Install Maven or use the Maven wrapper

33. Learning Outcomes

- Controller–Service–Repository architecture
- Spring dependency injection and IoC
- REST API development
- Spring Data JPA and automatic CRUD
- PostgreSQL database integration
- Validation with @Min and @NotNull
- Global exception handling
- Transactional business logic using @Transactional and row locking
- DSA in practice: priority queue and greedy slot assignment
- Postman API testing
- FastAPI-to-Spring Boot microservice communication

34. Final Command Sheet

```
# Verify Java
java -version

# Verify Maven
mvn -version

# Build
mvn clean install
```

```
# Run
mvn spring-boot:run

# API
GET http://localhost:8080/inventory/101
POST http://localhost:8080/orders
GET http://localhost:8080/slotting/plan?slots=A-01,A-02,A-03

# PostgreSQL
CREATE DATABASE iwms_week8;

# Verify data
SELECT * FROM inventory;
SELECT * FROM stock_orders;
```

One More Example for Practice: Microservices Using Spring Boot

This example shows service discovery with Eureka. A Report Service calls the Inventory Stock Service by name instead of by URL.

Part A – Creation of the Eureka Server

1. In Spring Initializr (or Spring Tool Suite) create a Maven project named `EurekaServer` with Java 17, group `com.iwms`, and the dependencies **Eureka Server**, **Spring Boot DevTools** and **Spring Web**.
2. Open the main class and add `@EnableEurekaServer` below `@SpringBootApplication`.
3. Rename `application.properties` to `application.yml` and add:

```
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false

server:
  port: 8761

spring:
  application:
    name: EUREKA-SERVER
```

4. Run the project and open `http://localhost:8761`. The Eureka dashboard should appear with no instances registered yet.

Part B – Stock Service (Eureka client)

Create a project `StockService` with the dependencies **Eureka Discovery Client**, **Spring Boot DevTools** and **Spring Web**. Add `@EnableDiscoveryClient` to the main class.

```
@RestController
@CrossOrigin
public class StockController {

    @GetMapping("/stock/{productId}")
    public String getStock(@PathVariable Long productId) {
```

```

        return "Product " + productId + " stock: 45 units";
    }
}

```

```

eureka:
  client:
    registerWithEureka: true
    fetchRegistry: true
    serverUrl:
      defaultZone: http://localhost:8761/eureka/
  instance:
    hostname: localhost
server:
  port: 9090
spring:
  application:
    name: INVENTORY-STOCK-SERVICE

```

Part C – Report Service calls Stock Service through Eureka

Create a project `ReportService` with the same three dependencies. Add a load-balanced `RestTemplate` to the main class:

```

@SpringBootApplication
@EnableDiscoveryClient
public class ReportServiceApplication {

    @Bean
    @LoadBalanced
    public RestTemplate getRestTemplate() {
        return new RestTemplate();
    }

    public static void main(String[] args) {
        SpringApplication.run(ReportServiceApplication.class, args);
    }
}

```

```

@RestController
@CrossOrigin("")
public class ReportController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/report/{productId}")
    public String stockReport(@PathVariable Long productId) {
        String url = "http://INVENTORY-STOCK-SERVICE/stock/" + productId;
        return restTemplate.getForObject(url, String.class);
    }
}

```

```
eureka:
  client:
    registerWithEureka: true
    fetchRegistry: true
    serverUrl:
      defaultZone: http://localhost:8761/eureka/
  instance:
    hostname: localhost
server:
  port: 8001
spring:
  application:
    name: REPORT-SERVICE
```

Part D – Run and verify

1. Start the Eureka Server, then Stock Service, then Report Service.
2. Refresh <http://localhost:8761>. Both **INVENTORY-STOCK-SERVICE** (port 9090) and **REPORT-SERVICE** (port 8001) should show status UP.
3. In Postman call GET <http://localhost:8001/report/101>. Expected response: `Product 101 stock: 45 units`.
4. Take screenshots of the Eureka dashboard and the Postman response for your record.